

Errori nei dati di training: impatto sui modelli di Machine Learning

Descrizione tecnica dei notebook Teen Mental Health, Adult Income e Liver Patient

Tommaso Baggio

Samuele Bergamin

Andrea Ciacchi

Anno accademico 2025–2026

Indice

1	Obiettivo del progetto	3
2	Correzioni metodologiche comuni	3
2.1	Separazione tra training, validation e test	3
2.2	Metriche	3
2.3	Ensemble di Decision Tree	4
2.4	Rete neurale	4
2.5	Support Vector Machine	4
3	Generazione e controllo del rumore	4
3.1	Feature categoriche one-hot	5
3.2	Feature continue	5
3.3	Feature intere e ordinali	5
3.4	Selezione delle feature da corrompere	5
4	Teen Mental Health Dataset	5
4.1	Dati e target	5
4.2	Preprocessing	6
4.3	Flusso sperimentale	6
4.4	Correzioni specifiche	6
5	Adult Income Dataset	6
5.1	Dati e target	6
5.2	Preprocessing	7
5.3	Flusso sperimentale e correzioni	7

6	Liver Patient Dataset	7
6.1	Dati e target	7
6.2	Preprocessing e rumore	7
6.3	Coerenza del target e validazione	8
7	Riproducibilità e interpretazione dei risultati	8
8	Conclusioni	8

1 Obiettivo del progetto

Il progetto studia come la corruzione delle feature del training set modifichi le prestazioni di modelli di classificazione binaria. L'analisi viene ripetuta su tre dataset con caratteristiche molto diverse:

1. **Teen Mental Health Dataset**, piccolo e fortemente sbilanciato;
2. **Adult Income Dataset**, più grande e con numerose variabili categoriche;
3. **Liver Patient Dataset**, piccolo, clinico e caratterizzato da forti outlier.

Per ogni dataset vengono confrontati tre modelli: ensemble di Decision Tree, ensemble di reti neurali e ensemble di Support Vector Machine. Il rumore viene aggiunto esclusivamente al training set; validation e test rimangono puliti. In questo modo si misura la perdita di capacità di generalizzazione causata da errori nei dati usati per apprendere.

2 Correzioni metodologiche comuni

I notebook sono stati revisionati per eliminare diverse sorgenti di risultati ottimistici o incoerenti.

2.1 Separazione tra training, validation e test

Il dataset viene prima diviso in training+validation e test mediante uno split stratificato. Il blocco training+validation viene poi diviso nuovamente, sempre in modo stratificato. Nel flusso principale le proporzioni sono circa:

70% training, 15% validation, 15% test.

Il training serve per stimare i parametri del modello. La validation serve per scegliere profondità, potatura e pesi di classe. Il test viene consultato solo dopo la selezione. Questa modifica elimina il *test leakage*: scegliere gli iperparametri sul test renderebbe la valutazione finale eccessivamente favorevole.

2.2 Metriche

Oltre all'accuracy vengono calcolate:

- precision e recall della classe positiva;
- F1-score della classe positiva;
- macro-F1, che assegna uguale importanza alle classi;
- balanced accuracy, media del recall delle due classi;
- matrice di confusione.

Le chiamate a `classification_report` ora rispettano l'ordine corretto (`y_true`, `y_pred`). In precedenza alcuni notebook invertivano questi argomenti, scambiando il significato di precision e recall.

2.3 Ensemble di Decision Tree

Per ogni configurazione vengono costruiti tre alberi. Uno `StratifiedKFold` a tre fold produce, a ogni iterazione, gli indici dei due fold di training e del fold escluso. Gli alberi vengono correttamente addestrati sugli indici di training, cioè su circa due terzi dei dati. Le tre predizioni vengono combinate tramite voto di maggioranza.

La ricerca considera:

- `max_depth`, per limitare la complessità;
- `ccp_alpha`, per il cost-complexity pruning;
- `class_weight`, per gestire lo sbilanciamento.

Il criterio di potatura può essere scritto come

$$R_\alpha(T) = R(T) + \alpha|T|,$$

dove $R(T)$ è l'errore dell'albero e $|T|$ il numero di foglie. La configurazione viene scelta sulla validation, mentre il test resta separato.

2.4 Rete neurale

La rete è composta da due livelli nascosti densi con 32 e 16 neuroni ReLU e da un neurone sigmoide di output. La loss è la binary cross-entropy. Le feature vengono trasformate con `RobustScaler`, stimato esclusivamente sul training corrente. Nei test progressivi vengono usati:

- bootstrap del training per diversificare i modelli;
- pesi di classe calcolati sul training;
- early stopping sulla validation loss;
- media delle probabilità dei modelli e soglia 0.5;
- ripetizioni con seed controllati e deviazione standard delle metriche.

2.5 Support Vector Machine

La SVM usa kernel RBF, `C=1`, `gamma="scale"` e `class_weight="balanced"`. Anche in questo caso il `RobustScaler` viene stimato soltanto sul training. Tre modelli vengono addestrati su bootstrap riproducibili e combinati tramite voto di maggioranza.

3 Generazione e controllo del rumore

I livelli analizzati sono 0%, 10%, ..., 100%. Per ogni livello si riparte dal training pulito, evitando che il rumore si accumuli tra un livello e il successivo. I seed sono deterministici, così il confronto è riproducibile.

3.1 Feature categoriche one-hot

Una variabile categorica viene codificata tramite un gruppo di dummy. La corruzione non modifica le dummy separatamente: identifica la categoria attiva, la disattiva e attiva una categoria alternativa. Viene verificato che per ogni riga valga

$$\sum_{k \in G} x_{ik} = 1,$$

dove G è il gruppo di dummy della variabile. Si evitano così combinazioni impossibili con zero o più categorie simultaneamente attive.

3.2 Feature continue

Su una frazione ρ delle righe viene aggiunto rumore gaussiano:

$$\tilde{x}_{ij} = x_{ij} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, (\lambda s_j)^2),$$

dove s_j è la deviazione standard della feature nel training e λ è `noise_level`. Quando il dominio lo richiede viene applicato il clipping, per esempio per impedire valori clinici negativi.

3.3 Feature intere e ordinali

Le variabili intere non vengono trasformate in numeri decimali. Viene scelta una variazione locale tra $-2, -1, +1, +2$ e il risultato viene mantenuto entro limiti espliciti. Nei notebook non si usa più la regola fragile “intero con pochi valori uguale categorico”: il tipo di ogni feature è dichiarato nello schema del dataset.

3.4 Selezione delle feature da corrompere

La Mutual Information viene calcolata solo sul training. Le dummy vengono prima ricondotte alla feature categorica originale, che riceve un unico valore di importanza. Le feature vengono ordinate e divise in due gruppi, migliori e peggiori. Per un numero dispari, la feature centrale può appartenere a entrambi i gruppi, coerentemente con il criterio adottato nei notebook.

4 Teen Mental Health Dataset

4.1 Dati e target

Il dataset contiene 1200 adolescenti e 12 feature: età, genere, uso dei social, piattaforma, sonno, uso dello schermo prima di dormire, rendimento accademico, attività fisica, interazione sociale, stress, ansia e dipendenza. Il target `depression_label` vale 1 per la classe positiva.

La classe è estremamente sbilanciata: 31 positivi contro 1169 negativi. Lo split stratificato è quindi indispensabile. Anche con la stratificazione validation e test contengono pochi positivi, perciò accuracy e singole confusion matrix devono essere interpretate con cautela.

4.2 Preprocessing

`gender`, `platform_usage` e `social_interaction_level` vengono convertite con one-hot encoding. Lo schema esplicito distingue:

- nominali: genere, piattaforma e interazione sociale;
- ordinali: stress, ansia e dipendenza, limitate tra 1 e 10;
- intera: età, limitata tra 13 e 19;
- continue: ore sui social, sonno, schermo prima del sonno, rendimento e attività fisica.

Le feature continue sono limitate a domini ammissibili: ore e attività tra 0 e 24, rendimento tra 0 e 4. La corruzione delle ordinali usa variazioni intere locali, mentre le nominali vengono modificate per gruppo one-hot.

4.3 Flusso sperimentale

Il notebook addestra inizialmente un albero non potato e uno con profondità limitata. Successivamente calcola il percorso di cost-complexity pruning e valuta su validation combinazioni di profondità, alpha e pesi. Viene poi costruito l'ensemble di tre alberi.

La Mutual Information è stimata sul training e aggregata per feature originale. Le liste migliori e peggiori vengono usate negli esperimenti progressivi per alberi, NN e SVM. Il confronto puntuale tra dataset pulito e rumoroso crea realmente una copia `X_train_noisy`; prima della revisione il codice riaddestrava il modello sullo stesso training pulito, producendo un confronto privo di significato.

4.4 Correzioni specifiche

Sono stati corretti: ordine instabile delle feature, codifica ordinale impropria delle categoriche, classificazione automatica basata su `nunique`, corruzione indipendente delle dummy, Mutual Information frammentata e uso del test durante il tuning. Il notebook non applica undersampling attivo. Rimane un limite intrinseco: il target è quasi determinato da combinazioni di social media, sonno, stress e ansia, quindi prestazioni molto elevate possono riflettere la costruzione del dataset più che una generalizzazione clinica.

5 Adult Income Dataset

5.1 Dati e target

Il dataset contiene 48842 righe. Il target `income` viene mappato esplicitamente come `<=50K` → 0 e `>50K` → 1. La colonna testuale `education` viene rimossa perché ridondante rispetto a `educational-num`.

Nei campi `workclass`, `occupation` e `native-country`, il simbolo “?” rappresenta un valore mancante. Viene convertito nella categoria esplicita `Unknown`, evitando che un codice numerico arbitrario nasconda il significato del dato mancante.

5.2 Preprocessing

Le variabili `workclass`, `marital-status`, `occupation`, `relationship`, `race`, `native-country` e `gender` vengono codificate one-hot. Il target usa un mapping esplicito e viene verificato che non produca valori nulli.

Lo schema distingue:

- feature nominali: i sette gruppi one-hot;
- intere: `age`, `educational-num`, `hours-per-week`;
- continue: `fnlwgt`, `capital-gain`, `capital-loss`.

Le feature intere vengono perturbate localmente e mantenute rispettivamente negli intervalli 17–90, 1–16 e 1–99. Le categoriche vengono cambiate come gruppi one-hot validi.

5.3 Flusso sperimentale e correzioni

Il notebook segue la pipeline comune: split stratificato, ricerca degli iperparametri su validation, valutazione finale su test, ensemble di alberi, rete neurale e SVM. La Mutual Information viene calcolata sul training e ricostruisce una sola variabile discreta per ciascun gruppo one-hot.

Sono stati eliminati: `cat.codes`, il nome errato `Gender`, il riferimento a `education` dopo la sua rimozione, i residui `SGPT/AG ratio`, il bug dei fold dell'ensemble e il confronto rumoroso che non applicava rumore. Il nome corretto della colonna numerica è `educational-num`; lo schema è stato verificato direttamente sul CSV.

Il confronto pulito/rumoroso usa lo stesso split e modifica soltanto il training. Le pipeline progressive ricostruiscono i gruppi migliori e peggiori sul training interno, lasciando puliti validation e test.

6 Liver Patient Dataset

6.1 Dati e target

Il dataset contiene 583 righe e le feature `Age`, `Gender`, bilirubina totale e diretta, fosfatasi alcalina, transaminasi SGPT e SGOT, proteine totali, albumina e rapporto albumina/globuline. Il mapping è unico in tutto il notebook:

`Liver Disease` → 1, `No Liver Disease` → 0.

Sono presenti 416 pazienti con malattia e 167 senza malattia. La classe 0 è quindi minoritaria. I pesi manuali aumentano correttamente il costo degli errori sulla classe 0; prima della revisione alcune sezioni aumentavano invece il peso della classe 1, già maggioritaria.

6.2 Preprocessing e rumore

`Gender` viene convertita in due dummy one-hot. `Age` è una feature intera e viene perturbata localmente, restando tra 0 e 120. Le variabili cliniche sono continue e il rumore viene limitato inferiormente a zero, impedendo concentrazioni biochimiche negative.

La Mutual Information usa lo stesso training dei modelli e ricostruisce **Gender** come singola variabile discreta. Le liste migliori e peggiori vengono successivamente espanse nelle dummy solo quando serve addestrare il modello o corrompere il training.

6.3 Coerenza del target e validazione

Le sezioni NN e SVM non applicano più la trasformazione 1 - **Selector**. Tale operazione invertiva la semantica della classe positiva soltanto in alcuni esperimenti: F1 e recall della classe 1 indicavano quindi concetti diversi nelle varie sezioni. Ora 1 significa sempre malattia epatica.

Alberi iniziali e potati vengono osservati sulla validation. Profondità, alpha e pesi sono selezionati sulla validation; il test resta separato. Anche il confronto pulito/rumoroso costruisce realmente `X_train_noisy` e usa lo stesso validation set.

7 Riproducibilità e interpretazione dei risultati

Tutte le operazioni casuali principali ricevono un `random_state`. Lo scaler è stimato dopo la corruzione, solo sul training del livello corrente. I bootstrap usano seed distinti ma riproducibili. Le percentuali di rumore partono sempre dalla copia pulita del training.

Le curve devono essere interpretate considerando contemporaneamente accuracy, balanced accuracy, precision, recall e F1. Una metrica stabile non implica necessariamente robustezza: il modello potrebbe modificare il compromesso tra falsi positivi e falsi negativi. Nei dataset piccoli, soprattutto Teen e Liver, è inoltre opportuno considerare la variabilità tra split e ripetizioni.

8 Conclusioni

La revisione ha trasformato i notebook da sequenze esplorative dipendenti dallo stato in pipeline più coerenti. Le modifiche principali sono: separazione corretta dei dati, mapping esplicito dei target, one-hot encoding, scelta dei pesi coerente con la classe minoritaria, Mutual Information calcolata sul training, rumore valido rispetto al dominio e confronti pulito/rumoroso realmente controllati.

L'esperimento permette ora di attribuire con maggiore affidabilità le variazioni delle metriche alla corruzione del training set, invece che a leakage, encoding arbitrari, inversioni del target o split differenti.